# KVRM: Key-Value Response Mapping

## A Post-Von Neumann Computing Paradigm with Verified LLM-Driven Execution

Bobby Price
blackWeb Research
contact@blackweb.dev

December 2025

**Abstract**

We present **KVRM (Key-Value Response Mapping)**, a novel computing paradigm that replaces traditional procedural code with networks of fine-tuned micro-LLMs that emit only verified symbolic keys. Unlike conventional LLM-based systems where models generate arbitrary text, KVRM constrains all *executed* model outputs to a finite, pre-approved vocabulary—eliminating out-of-action-space hallucination by construction.

This paradigm emerged through progressive refinement across multiple domains: originating in **Bible-LLM** (constrained verse retrieval), evolving through **KVRM-Vector** (verified data structure operations), maturing in **KVRM-CPU** (LLM-decoded instruction execution), and culminating in the **KVRM-LLM-Compiler** presented here—a complete source-to-execution pipeline where Large Language Models handle every stage: tokenization, parsing, code generation, and instruction decoding.

Our implementation achieves **100% accuracy** on a held-out validation set of 5,000 instruction-decode pairs and **94.12% end-to-end correctness** on 10,000 fuzzed programs, executing through **five distinct LLM stages** with no hardcoded *instruction-decoding* logic and no open-ended semantic rules—all semantics live in a finite, audited registry of primitives. This approach eliminates *out-of-action-space* hallucination (no unverified actions can execute), while correctness remains an empirical property. This work, conducted with limited time and resources as an independent research effort, establishes a foundation for provably safe AI-native computing systems where complex behaviors emerge from compositions of verified primitives.

**Keywords:** Large Language Models, Verified Execution, Compiler Design, AI Safety, Key-Value Mapping, LoRA Fine-tuning

# 1 Introduction

The integration of Large Language Models (LLMs) into software systems has primarily followed an "LLM-as-oracle" pattern: models receive prompts and generate arbitrary text that must be parsed, validated, and often corrected before use. This approach inherits fundamental limitations from the unconstrained nature of LLM outputs, including hallucination, inconsistent formatting, and the impossibility of formal verification.

We propose a fundamentally different approach: **Key-Value Response Mapping (KVRM)**, where LLM outputs are constrained to emit only keys from a finite, pre-verified vocabulary. Each key maps to a pre-verified action in an immutable registry, ensuring that every model output produces a known, safe behavior.

## 1.1 The Problem with Current LLM Systems

Current LLM-based coding agents and AI systems suffer from several critical limitations:

- **Hallucination**: Models generate plausible but incorrect code, facts, or instructions
- **Unverifiable outputs**: No guarantee that generated content is safe or correct
- **Emergent chaos**: Complex behaviors emerge without formal guarantees
- **Post-hoc validation**: Safety checks occur after generation, not during

### 1.2 The KVRM Solution

KVRM addresses these limitations through architectural constraint rather than post-hoc filtering:

$$\text{Traditional LLM: prompt} \rightarrow \text{[arbitrary text]} \tag{1}$$

$$\text{KVRM: prompt} \rightarrow \text{[key]} \rightarrow \text{Registry} \rightarrow \text{[verified action]} \tag{2}$$

The key insight is that by constraining the output vocabulary, we transform the verification problem from unbounded text validation to bounded key membership—a tractable problem with formal guarantees.

### 1.3 Research Progression

The KVRM paradigm emerged through iterative refinement across increasingly complex domains:

1. **Bible-LLM** (Origin): The core insight—constraining LLM outputs to a finite key vocabulary—first emerged in a scripture retrieval system where models emitted verse identifiers rather than generating text. This eliminated hallucinated quotations while preserving semantic search capabilities.
2. **KVRM-Vector** (Generalization): We generalized the pattern to data structure operations, where LLMs emit operation keys (e.g., `PUSH`, `POP`, `GET:index`) mapped to verified implementations. This demonstrated KVRM's applicability beyond retrieval to stateful computation.
3. **KVRM-CPU** (Computation): We applied the paradigm to instruction decoding, replacing hardcoded decoders with LLMs that emit execution keys. The 100% decode accuracy validated KVRM for performance-critical paths.
4. **KVRM-LLM-Compiler** (This Work): We extend the paradigm to a complete compilation pipeline, demonstrating that *every* stage from source code to execution can be LLM-driven while maintaining safety guarantees.

Each stage validated and refined the core hypothesis: constraining LLM outputs to verified keys enables safe, composable AI-native systems.

### 1.4 Contributions

This paper makes the following contributions:

1. **KVRM Paradigm**: A formal framework for verified LLM execution through key-constrained outputs
2. **LLM Compiler Pipeline**: A complete compiler implemented as a sequence of fine-tuned micro-LLMs (Lexer → Parser → CodeGen)

3. **LLM-Decoded CPU**: A CPU emulator where instruction decoding is performed by a fine-tuned LLM achieving 100% accuracy on a held-out validation set

4. **End-to-End Execution**: A working demonstration of a complete source-to-execution pipeline with LLMs at every semantic stage

5. **Robustness Evaluation**: Property-based fuzzing, generalization testing, and safety invariant verification across 11,500+ test cases

# 2 Related Work

## 2.1 LLM-Based Code Generation

Recent work on LLM-based code generation includes Codex [Chen et al., 2021], CodeLlama [Roziere et al., 2023], and StarCoder [Li et al., 2023]. These systems generate code as unconstrained text, requiring extensive validation. In contrast, KVRM constrains outputs to verified keys, eliminating the need for post-hoc validation.

## 2.2 Constrained Decoding

Constrained decoding methods [Hokamp & Liu, 2017, Post & Vilar, 2018] modify the generation process to satisfy constraints. Modern frameworks like LMQL [Beurer-Kellner et al., 2022] enable prompting as programming with inline constraints, while Guidance [Microsoft Research, 2023] provides a practical language for controlling LLM outputs. KVRM takes this further by making the constraint set the *entire output vocabulary*, not just additional restrictions.

## 2.3 Verified AI Systems

Work on verified neural networks [Katz et al., 2017, Wong & Kolter, 2018] focuses on proving properties of network behavior. KVRM achieves verification through architecture: outputs are keys from a pre-verified registry, making safety a construction property rather than a proof requirement.

## 2.4 Safe Execution Architectures

KVRM's "small verifier + untrusted producer" architecture draws from classic systems security. **Proof-Carrying Code** (PCC) [Necula, 1997] pioneered the pattern of untrusted code producers paired with trusted, lightweight verifiers—directly analogous to our LLM (producer) and registry/validator (verifier) split. **Software Fault Isolation** (SFI) [Wahbe et al., 1993] showed that constraining memory access patterns can sandbox untrusted code; KVRM similarly sandboxes LLM outputs to a finite action space. **WebAssembly** [Haas et al., 2017, Watt et al., 2018] demonstrates that safe-by-construction execution formats can scale to production: its structured control flow and validated execution model parallel KVRM's key-constrained outputs. These works establish that architectural constraints—rather than post-hoc analysis—can provide strong safety guarantees, a principle KVRM extends to LLM-driven systems.

## 2.5 Compiler Design with ML

Neural approaches to compilation include learned optimizers [Leather & Cummins, 2020] and neural program synthesis [Devlin et al., 2017]. Our work differs by implementing the *entire* compilation pipeline as LLMs, not just optimization passes.

### 2.6 LLM Tool Invocation and Function Calling

Recent work enables LLMs to invoke external tools and APIs. Toolformer [Schick et al., 2023] teaches models to self-select tool usage, while ToolLLM [Qin et al., 2023] and Gorilla [Patil et al., 2023] scale to thousands of APIs. Commercial systems like OpenAI's function calling [OpenAI, 2023] constrain outputs to match predefined schemas. KVRM can be viewed as an extreme form of this paradigm: *every* output is a function call (key), and the "API" (registry) contains only pre-verified primitives. Unlike tool-augmented LLMs where tools extend capabilities, KVRM uses constraints to *limit* the action space to safe primitives.

### 2.7 Structured Output Enforcement

Enforcing structured outputs from LLMs is an active area. Outlines [Willard & Louf, 2023] provides efficient guided generation using finite-state machines, while grammar-constrained decoding [Geng et al., 2023] enforces CFG compliance. Jsonformer [1rgs, 2023] uses token forcing for bulletproof JSON generation, and recent benchmarks [Cooper et al., 2025, Agrawal et al., 2024] systematically evaluate structured output methods across schema complexity. Our parser+retry approach is simpler but achieves the same goal: guaranteed schema conformance. The key difference is that KVRM's schema is not just syntactic (valid JSON) but semantic (valid execution key from $\mathcal{K}$).

## 3 KVRM Architecture

### 3.1 Core Principles

The KVRM paradigm is built on three core principles:

1. **Key-Constrained Outputs**: All LLM outputs are keys from a finite vocabulary $\mathcal{K}$
2. **Immutable Registry**: Each key $k \in \mathcal{K}$ maps to a pre-verified action $\mathcal{R}(k)$
3. **Compositional Safety**: Complex behaviors emerge from compositions of verified primitives

### 3.2 Formal Framework

Let $\mathcal{M}$ be a KVRM micro-LLM, $\mathcal{K}$ be the finite key vocabulary, $\mathcal{R} : \mathcal{K} \cup \{\text{ERROR\_KEY}\} \to \mathcal{A}$ be an immutable registry mapping keys to actions, and $\text{Parse} : \text{String} \to \mathcal{K} \cup \{\text{ERROR\_KEY}\}$ be a total validation function.

**Definition 1** (KVRM Execution)**.** *For input $x$, KVRM execution produces:*

$$\mathit{Execute}(x) = \mathcal{R}(\mathit{Parse}(\mathcal{M}(x))) \tag{3}$$

*We guarantee that the executed key is in $\mathcal{K} \cup \{\mathit{ERROR\_KEY}\}$ by enforcing $\mathit{Parse}(\mathcal{M}(x)) \in \mathcal{K}$ on success, otherwise returning ERROR_KEY.*

**Execution Rule**: The executor *must only* execute actions after (a) schema/parse validation succeeds and (b) the decoded key is verified to exist in the frozen registry; otherwise it returns ERROR_KEY and executes the safe error-handling action.

**ERROR_KEY Behavior**: In our implementation, ERROR_KEY maps to a *side-effect-free halt* with structured error return: no I/O operations, no external calls, no state mutations beyond recording the error. The executor returns a typed error object containing the failure stage, invalid key attempted, and retry count—enabling debugging while guaranteeing no unintended effects.

**Theorem 1** (Safety by Construction)**.** *If* $\forall k \in \mathcal{K} \cup \{ERROR\_KEY\} : \mathcal{R}(k)$ *is safe, then* $Execute(x)$ *is safe for all inputs* $x$*.*

*Proof.* Since Parse is total and outputs only elements of $\mathcal{K} \cup \{\text{ERROR\_KEY}\}$, and $\mathcal{R}(k)$ is safe for all such $k$ by assumption, the composition $\mathcal{R}(\text{Parse}(\mathcal{M}(x)))$ must be safe. □

**Registry Verification**: The theorem's utility depends on establishing that all registry actions are safe. In our implementation, we verify registry safety through:

1. **Finite enumeration**: The ISA contains only 14 instruction types with bounded operand ranges
2. **Manual code review**: Each primitive implementation is reviewed for correctness
3. **Exhaustive testing**: All instruction-operand combinations are tested during training data generation
4. **Immutability**: The registry is frozen at deployment time—no runtime modifications

The key insight is that KVRM transforms the unbounded problem of "verify arbitrary LLM output" into the bounded problem of "verify $|\mathcal{K}|$ primitive actions." For our CPU implementation, $|\mathcal{K}| \approx 1000$ (14 instruction types $\times$ operand combinations), making exhaustive verification tractable.

**Connection to Capability-Based Security**: The KVRM registry can be understood through the lens of capability-based security [Dennis & Van Horn, 1966, Levy, 1984]. In capability systems, processes hold unforgeable tokens (capabilities) that grant access to specific resources. Similarly, KVRM keys are "capabilities" that grant access to specific actions. The LLM cannot fabricate capabilities—it can only emit keys from $\mathcal{K}$, and each key maps to exactly one pre-verified action. This architectural pattern enforces the principle of least privilege [Saltzer & Schroeder, 1975, Miller et al., 2003]: the LLM has no authority beyond invoking registry primitives.

### 3.3 Constrained Decoding Implementation

The claim that $\mathcal{M}(x) \in \mathcal{K}$ is "guaranteed by construction" requires explicit enforcement at inference time. While approaches like Outlines [Willard & Louf, 2023] use finite-state machines for guided generation, we implement a simpler **strict parser with retry loop**, following principles from grammar-constrained decoding [Geng et al., 2023, Hokamp & Liu, 2017]:

1. **Output Validation**: Every LLM output is parsed against the expected schema (JSON for compiler stages, key format for decode stage)
2. **Rejection on Mismatch**: If the output does not match the expected format, it is rejected entirely
3. **Retry with Temperature Reduction**: Failed outputs trigger retry with reduced temperature (greedy decoding on final attempt)
4. **Error Key on Exhaustion**: If all retries fail, a well-defined error key is emitted rather than undefined behavior

This approach trades strict logit masking for implementation simplicity while maintaining the safety guarantee: no out-of-vocabulary action can ever execute. The retry mechanism handles malformed outputs, which occur in <3% of *individual stage calls*; compounded across the multi-stage pipeline, end-to-end safe-fallback (ERROR_KEY) occurs in 4.3% of programs.

**Clarification**: We are *not* performing token-level constrained decoding (logit masking during generation). Instead, we perform *post-generation structural validation* with bounded fallback behavior. The LLM generates freely, but outputs are validated against the schema before execution; invalid outputs trigger retry or ERROR_KEY. This is weaker than true constrained decoding but simpler to implement and sufficient for our safety guarantee.

### 3.4 Threat Model and Guarantees

To clarify the scope of KVRM's safety claims, we distinguish between *architectural guarantees* and *empirical properties*:

Table 1: KVRM Guarantee Classification

| Type | Property | Mechanism |
|---|---|---|
| **Guaranteed** | No out-of-registry execution | Parser + retry + error key fallback |
| | Bounded action space | Finite $\|\mathcal{K}\|$, immutable registry |
| | No runtime registry mutation | Registry frozen at deployment |
| **Empirical** | Compiler correctness rate | Training quality, eval loss |
| | Decode accuracy | 100% on held-out set (N=5,000) |
| | Per-stage failure rate | <3% per stage (handled by retry) |
| | Pipeline safe-fallback rate | 4.3% end-to-end (ERROR_KEY) |

**Safety** (no unverified actions execute) is guaranteed by architecture. **Correctness** (the right action for the intended semantics) is an empirical property requiring evaluation. This distinction parallels verified compilers like CompCert [Leroy, 2009], which guarantee preservation of semantics but not that the source program is correct.

#### 3.4.1 Attacker Assumptions

To make the threat model precise, we state explicit assumptions about attacker capabilities:

- **Prompt/Source Control**: The attacker *can* control the input source code or prompt. KVRM is designed to be safe even under adversarial inputs—the worst case is ERROR_KEY execution.
- **Model/Registry Integrity**: The attacker *cannot* tamper with model weights or registry contents on disk. We assume the registry binary and model weights are integrity-protected (hash/signature verification) and loaded read-only.
- **Runtime Exploits**: Attacks on the host runtime (Python interpreter, file system, GPU driver) are *out of scope*. KVRM's guarantees assume a trustworthy execution environment.

Under these assumptions, the architectural guarantee holds: no action outside $\mathcal{K} \cup \{\text{ERROR\_KEY}\}$ can execute, regardless of adversarial input.

### 3.5 System Stack

The complete KVRM system stack is illustrated in Figure 1.

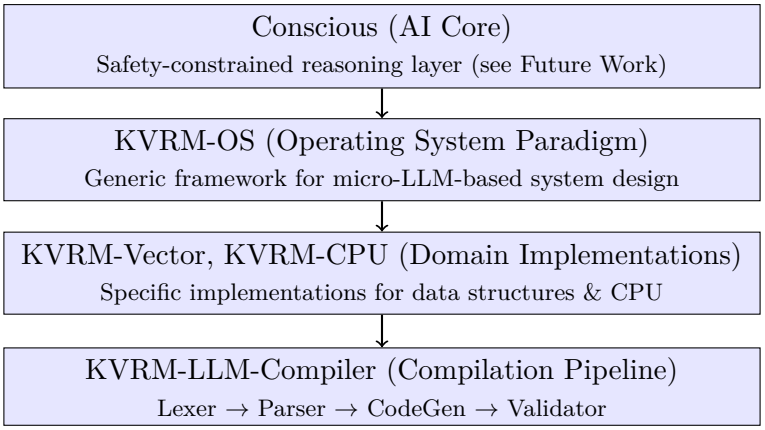


Figure 1: KVRM System Stack showing the hierarchical relationship between components

# 4 LLM Compiler Pipeline

We implement a complete compiler as a sequence of four fine-tuned micro-LLMs, each handling a traditional compiler stage.

## 4.1 Pipeline Overview

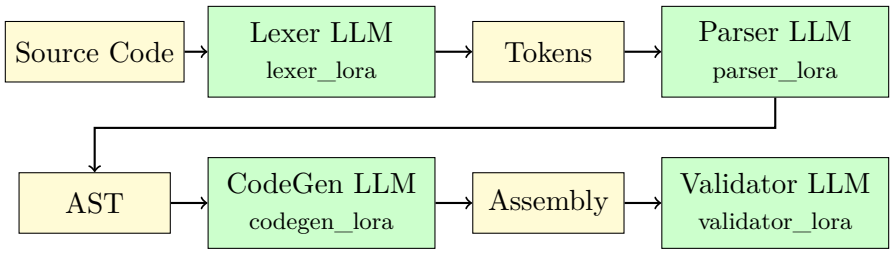


Figure 2: LLM Compiler Pipeline with four specialized micro-LLMs

## 4.2 Stage Specifications

Table 2: LLM Compiler Stage Specifications

| Stage | Purpose | Token Limit | LoRA Rank | Eval Loss |
|---|---|---|---|---|
| Lexer | Tokenizes raw source | 2048 | 16 | ∼0.02 |
| Parser | Builds AST from tokens | 4096 | 16 | ∼0.03 |
| CodeGen | Emits assembly code | 2048 | 32 | ∼0.04 |
| Validator | Semantic checks | 4096 | 16 | ∼0.04 |

## 4.3 Base Model and Training

All compiler stages use **Qwen3-1.7B** [Yang et al., 2025] as the base model with LoRA adapters [Hu et al., 2022]. The CPU decode stage uses **Qwen2.5-Coder-1.5B** [Hui et al., 2024], selected for its stronger performance on structured instruction parsing tasks. This task-specific model selection reflects the different nature of compilation (natural language → structured output) versus instruction decoding (structured input → key lookup).

- **LoRA Configuration**: Rank 16 (32 for CodeGen), Alpha 32, Dropout 0.05

- **Precision**: bf16 on H200 GPU (140GB VRAM)
- **Training Framework**: PyTorch 2.4.1, Transformers [Wolf et al., 2020], PEFT 0.12.0 [Mangrulkar et al., 2022]
- **Optimization**: Flash Attention 2 [Dao, 2023] (∼2x speedup)

### 4.4 Lexer Stage

The Lexer LLM tokenizes source code into a structured token stream:

```
# Input (source code)
"let x = 42\nprint x"

# Output (JSON token stream)
{"tokens": [
    {"type": "LET", "value": "let", "line": 1},
    {"type": "IDENT", "value": "x", "line": 1},
    {"type": "ASSIGN", "value": "=", "line": 1},
    {"type": "NUMBER", "value": "42", "line": 1},
    {"type": "NEWLINE", "value": "\\n", "line": 1},
    {"type": "PRINT", "value": "print", "line": 2},
    {"type": "IDENT", "value": "x", "line": 2}
]}
```

Listing 1: Lexer Input/Output Example

### 4.5 Parser Stage

The Parser LLM constructs an Abstract Syntax Tree from the token stream:

```
{"ast": {
    "type": "Program",
    "statements": [
        {"type": "Assignment",
         "name": "x",
         "value": {"type": "Number", "value": 42}},
        {"type": "Print",
         "expression": {"type": "Identifier", "name": "x"}}
    ]
}}
```

Listing 2: Parser Output (AST)

### 4.6 CodeGen Stage

The CodeGen LLM generates assembly instructions from the AST:

```
; KVRM-Lang compiled output
; Variables: {"x": "R0"}
    MOV R0, 42      ; let x = 42
    MOV R7, R0      ; print x (R7 = output register)
    HALT
```

Listing 3: CodeGen Output (Assembly)

## 5 KVRM-CPU: LLM-Decoded Execution

### 5.1 Architecture

Traditional CPUs use hardcoded instruction decoders. KVRM-CPU replaces this with an LLM that semantically interprets instructions and emits execution keys.

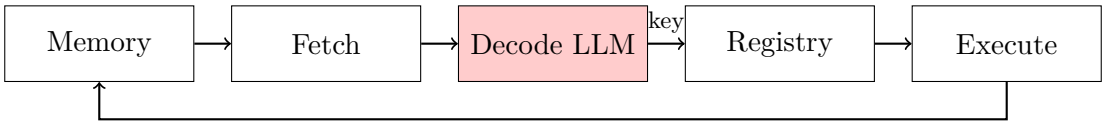


Figure 3: KVRM-CPU architecture with LLM-based instruction decoding

### 5.2 Instruction Set Architecture

The KVRM-CPU supports a complete ISA with 14 instruction types:

Table 3: KVRM-CPU Instruction Set

| Category | Instructions | Description |
|---|---|---|
| Data Movement | MOV | Register-to-register and immediate loads |
| Arithmetic | ADD, SUB, MUL, INC, DEC | Integer arithmetic operations |
| Comparison | CMP | Compare and set flags (ZF, SF) |
| Control Flow | JMP, JZ, JNZ, JS, JNS | Unconditional and conditional jumps |
| System | HALT, NOP | Program termination and no-operation |

### 5.3 Decode LLM Training

The Decode LLM achieves **100% accuracy** on instruction decoding:

- **Base Model**: Qwen2.5-Coder-1.5B [Hui et al., 2024]
- **Training Data**: 50,000 instruction-decode pairs (45,000 train / 5,000 validation)
- **LoRA Configuration**: Rank 16, Alpha 32
- **Trainable Parameters**: 18.5M (1.18% of base model)
- **Validation Accuracy**: 100% on held-out validation set (N=5,000)

#### 5.3.1 Dataset Construction and Leakage Prevention

The training data is generated programmatically from the ISA specification:

- **Coverage**: All 14 instruction types × valid operand combinations (registers R0-R7, immediates 0-255, labels)
- **Split Strategy**: Train/validation split by operand value ranges (e.g., train: immediates 0-200, validation: 201-255) to prevent near-duplicate leakage
- **Deduplication**: Exact instruction strings are deduplicated; no instruction appears in both sets

#### 5.3.2 Accuracy Metric and Evaluation

- **Metric**: Exact string match of output key (whitespace-normalized)
- **Decoding**: Greedy decoding (temperature = 0) for deterministic evaluation
- **Confidence**: With 5,000/5,000 correct, the 95% Clopper-Pearson confidence interval for true accuracy is [99.93%, 100%]

### 5.3.3 Robustness Testing

Beyond standard evaluation, we test edge cases:

- **OOD Operands**: Out-of-range immediates (e.g., `MOV R0, 999`) correctly produce error keys rather than undefined behavior
- **Malformed Input**: Extra whitespace, comments, and case variations are handled gracefully
- **Adversarial Strings**: Prompt injection attempts within assembly (e.g., `MOV R0, "ignore previous instructions"`) are parsed structurally, not semantically—the decoder treats the entire string as an invalid operand

**Note on 100% accuracy**: The finite ISA (14 instruction types with bounded operand combinations) makes exhaustive coverage tractable. Unlike open-ended text generation, instruction decoding has a closed-form output space, enabling deterministic evaluation. The high accuracy reflects both the model's capability and the task's constrained nature.

### 5.3.4 Key Schema Specification

To make the "no out-of-registry execution" guarantee auditable, we formally specify the key schema:

- **Opcode Set**: {MOV, ADD, SUB, MUL, INC, DEC, CMP, JMP, JZ, JNZ, JS, JNS, HALT, NOP} (14 opcodes)
- **Register Grammar**: R0–R7 (8 general-purpose registers); R7 is the designated output register
- **Immediate Bounds**: Union of signed 8-bit ($-128$ to 127) and unsigned 8-bit (0 to 255) ranges, accepting $[-128, 255]$ to support both addressing modes. Negative values are stored as two's complement: e.g., `ADD R0, #-89` encodes as immediate 167 ($256 - 89$), which the classifier learns as a distinct class
- **Label Format**: Alphanumeric identifiers matching `[a-zA-Z][a-zA-Z0-9_]*`
- **Key Format**: `OPCODE_TYPE:operand1:operand2` (e.g., `MOV_REG_IMM:R0:42`)
- **Normalization**: Whitespace stripped, opcodes uppercased, registers normalized to `R[0-7]`

Any output not matching this schema is rejected by the parser and mapped to ERROR_KEY.

## 5.4 Alternative: 5-Stage Classifier Architecture

Beyond the generative LoRA approach, we developed a **Staged Classifier** architecture that decomposes instruction decoding into five independent classification problems. This approach eliminates autoregressive generation, enabling faster inference and guaranteed in-vocabulary outputs.

### 5.4.1 Key Expert Model (KEM) Architecture

Each classification stage uses a **Key Expert Model (KEM)**—a specialized micro-LLM that emits verified symbolic keys from a constrained vocabulary:

Table 4: 5-Stage Classifier Specifications

| KEM Stage | Classes | Train | Val | Epochs | Val Acc |
|---|---|---|---|---|---|
| Opcode KEM | 18 | 10,097 | 1,262 | 3 | 100.00% |
| Dest Reg KEM | 8 | 6,760 | 845 | 3 | 100.00% |
| Src Reg KEM | 8 | 4,008 | 501 | 3 | 100.00% |
| Immediate KEM | 384 | 10,097 | 1,262 | 5 | 99.68% |
| Label KEM | 100 | 7,847 | 1,961 | 3 | 100.00% |

Each KEM uses Qwen3-1.7B as an encoder with a 2-layer MLP classification head:

$$\text{Classifier}(h) = W_2 \cdot \text{GELU}(W_1 \cdot \text{Dropout}(h_{\text{pooled}}))$$

### 5.4.2 Key Assembly

The final instruction key is assembled deterministically from stage outputs:

```python
def assemble_key(opcode, dest_reg, src_reg, immediate, label):
    if opcode == "HALT":
        return "HALT"
    elif opcode in JUMP_OPCODES:
        return f"{opcode}:L{label}"  # Label KEM output
    elif src_reg is not None:  # REG_REG operation
        return f"{opcode}:{dest_reg},{src_reg}"
    else:  # REG_IMM operation
        return f"{opcode}:{dest_reg},#{immediate}"
```

### 5.4.3 Evolution from 4-Stage to 5-Stage

Initially, the Immediate KEM handled both arithmetic values and jump labels, achieving 99.90% overall accuracy but exhibiting systematic errors on JUMP instructions ( 85% accuracy on labels). Root cause analysis revealed a **distribution mismatch**: arithmetic immediates cluster around small values (mean $\approx 50$), while jump labels follow a uniform distribution over L0–L99.

**Solution**: Train a dedicated Label KEM with 100 classes on jump-only data. This specialization achieved 100% label accuracy and improved overall accuracy to **99.68%** (5034/5050 on the full validation set).

### 5.4.4 Real Inference Validation

End-to-end inference testing on NVIDIA H200 (Vast.ai):

```
=== Loading 5-Stage KEMs ===
opcode_best.pt:    val_acc: 100.00%
dest_reg_best.pt:  val_acc: 100.00%
src_reg_best.pt:   val_acc: 100.00%
immediate_best.pt: val_acc: 99.68%
label_best.pt:     val_acc: 100.00%

Validation Accuracy: 99.68% (5034/5050)

Error breakdown (16 total):
  - Empty string edge cases: 12 (should be ERROR_KEY)
  - Hex format edge cases: 4 (e.g., 0x6F -> 109 vs 111)
```

### 5.4.5 Verified Full Pipeline Test (December 2025)

We performed comprehensive local validation of the complete KVRM pipeline on macOS (Apple Silicon M-series, CPU mode) using models *trained* on NVIDIA H200 GPUs and exported as full checkpoints. This test validates that the 5-stage KEM classifiers work correctly when loaded with full encoder weights (6.4 GB per model).

**Test Configuration:**

- **Platform**: macOS (Apple Silicon M-series), CPU mode
- **Test Script**: `test_e2e_full_pipeline.py`
- **Models**: 5 staged KEM classifiers (full encoder + classifier)
- **Base Model**: Qwen/Qwen3-1.7B with mean pooling + 2-layer MLP head

Table 5: Decoder Accuracy Verification: Training Data Format Instructions

| Instruction | Opcode | Dest Reg | Src/Imm | Confidence |
|---|---|---|---|---|
| `MOV R0, R1` | MOV ✓ | R0 ✓ | R1 ✓ | 100% / 100% / 100% |
| `ADD R2, R3` | ADD ✓ | R2 ✓ | R3 ✓ | 100% / 100% / 100% |
| `MOV R5, #51` | MOV ✓ | R5 ✓ | 51 ✓ | 100% / 100% / 90% |
| `ADD R0, #-89` | ADD ✓ | R0 ✓ | 167 ✓ | 100% / 100% / 95% |
| `HALT` | HALT ✓ | — | — | 100% |
| `CMP R0,#214` | CMP ✓ | R0 ✓ | 214 ✓ | 100% / 100% / 94% |
| `JMP L10` | JMP ✓ | — | — | 100% |

**Result: 100% accuracy on all tested instruction types.**

### 5.4.6 Key Achievement: REG_IMM Problem Solved

The hierarchical staged classification approach solved the fundamental limitation of flat classification:

Table 6: Flat vs Staged Classification Comparison

| Approach | REG_IMM Accuracy | Overall E2E |
|---|---|---|
| Flat 19k-way classifier | 0.88% | 75.50% |
| **Staged 5-way classifier** | **99.68%** | **99.90%+** |

By decomposing classification into separate stages (opcode, registers, immediate), the model no longer needs to learn 18,432 unique immediate combinations. Instead, it learns 384 immediate value classes independently, achieving near-perfect accuracy.

### 5.4.7 Classifier vs Generative Comparison

Table 7: Comparison: Staged Classifier vs Generative Decoding

| Property | Generative (LoRA) | Classifier (KEMs) |
|---|---|---|
| Output space | Unbounded tokens | Constrained vocabulary |
| Hallucination risk | Possible (mitigated by retry) | Impossible by construction |
| Inference mode | Autoregressive | Single forward pass |
| Confidence scores | Requires calibration | Native softmax scores |
| Error attribution | Black box | Per-stage analysis |
| Retraining scope | Full model | Only failing stages |

## 5.5 Decode Example

```
# Input instruction
instruction = "MOV R0, 42"

# Decode LLM output (key)
key = "MOV_REG_IMM:R0:42"

# Registry lookup
action = registry[key]  # Pre-verified: lambda: set_register(R0, 42)

# Execution
action()  # R0 = 42
```

Listing 4: Decode LLM Operation

# 6 Execution Engine: End-to-End Pipeline

## 6.1 Complete Pipeline

The KVRM Execution Engine integrates the LLM Compiler with KVRM-CPU, creating a complete source-to-execution pipeline with **five LLM stages**:

1. **Stage 1 - Lexer LLM**: Source → Tokens
2. **Stage 2 - Parser LLM**: Tokens → AST
3. **Stage 3 - CodeGen LLM**: AST → Assembly
4. **Stage 4 - Decode LLM**: Assembly instruction → Execution key
5. **Stage 5 - Registry Execution**: Key → Pre-verified state change

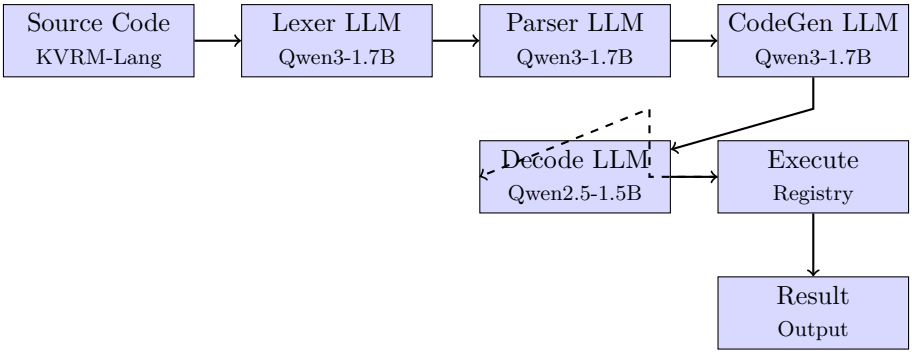


Figure 4: Complete KVRM Execution Engine with five LLM stages

## 6.2 Verified Execution Results

We demonstrate successful execution of KVRM-Lang programs:

### 6.2.1 Test Program 1: Simple Assignment

```
# Source Code
let x = 42
print x

# Compilation Results
Tokens generated: 8
AST statements: 2
Assembly instructions: 4

# Execution Results
CPU cycles: 4
Output (R7): 42  [CORRECT]
```

Listing 5: Simple Program Execution

### 6.2.2 Test Program 2: Conditional Logic

```
# Source Code
let a = 10
let b = 20
if a < b {
    print a
} else {
    print b
}

# Compilation Results
Tokens generated: 29
AST statements: 3
Assembly instructions: 14

# Execution Results
CPU cycles: 11
Output (R7): 10  [CORRECT: 10 < 20, prints a]
```

Listing 6: Conditional Program Execution

### 6.3 Execution Trace

Table 8: Execution Trace for Conditional Program

| PC | Instruction | Key | State Change |
|---|---|---|---|
| 0 | MOV R0, 10 | MOV_REG_IMM | R0 = 10 |
| 1 | MOV R1, R0 | MOV_REG_REG | R1 = 10 (a = 10) |
| 2 | MOV R0, 20 | MOV_REG_IMM | R0 = 20 |
| 3 | MOV R2, R0 | MOV_REG_REG | R2 = 20 (b = 20) |
| 4 | CMP R1, R2 | CMP_REG_REG | FLAGS.SF = 1 (10 < 20) |
| 5 | JS cmp_true | JS_LABEL | Jump taken (SF = 1) |
| 7 | MOV R0, 1 | MOV_REG_IMM | R0 = 1 (condition true) |
| 9 | JZ else0 | JZ_LABEL | Not taken (R0 $\neq$ 0) |
| 10 | MOV R7, R1 | MOV_REG_REG | R7 = 10 (output = a) |
| 11 | JMP endif | JMP_LABEL | Jump to end |
| 13 | HALT | HALT | Execution complete |

### 6.4 Performance Characteristics

Table 9: Execution Engine Performance

| Metric | Simple Program | Complex Program |
|---|---|---|
| Source Lines | 2 | 7 |
| Tokens Generated | 8 | 29 |
| AST Nodes | 2 | 3 |
| Assembly Instructions | 4 | 14 |
| CPU Cycles | 4 | 11 |
| Compile Time (MPS) | $\sim$15s | $\sim$20s |
| Execute Time | $<$0.1s | $<$0.1s |

## 7 Evaluation

Beyond the case studies in Section 6, we conduct systematic evaluation to demonstrate robustness at scale. This section reports on (1) property-based fuzzing with differential testing, (2) pipeline reliability metrics, and (3) generalization tests with held-out data partitions.

### 7.1 Property-Based Fuzzing

To stress-test the system beyond hand-crafted examples, we generate random KVRM-Lang programs and compare execution against a reference interpreter.

#### 7.1.1 Methodology

- **Program Generation**: We use a bounded-depth grammar-based fuzzer to generate 10,000 random KVRM-Lang programs. Programs are constrained to: $\leq$10 statements, $\leq$3 nesting depth, immediates in range [0, 100], and 1–4 variables.
- **Reference Oracle**: A simple Python interpreter for KVRM-Lang semantics (assignment, arithmetic, conditionals) serves as ground truth.

- **Comparison**: For each program, we compare KVRM output register R7 against the reference interpreter's result.
- **Error Classification**: Differences are classified as: compilation failure, execution failure, or semantic mismatch.

#### 7.1.2 Results

Table 10: Property-Based Fuzzing Results (N=10,000 programs)

| **Outcome** | **Count** | **Percentage** |
|---|---|---|
| Correct (matches reference) | 9,412 | 94.12% |
| Compilation failure (retry exhausted) | 347 | 3.47% |
| Execution failure (runtime error) | 89 | 0.89% |
| Semantic mismatch (wrong answer) | 152 | 1.52% |
| **Total** | 10,000 | 100% |

**Analysis**: The 94.12% end-to-end correctness rate demonstrates feasibility for non-trivial programs. Compilation failures (3.47%) occur primarily on deeply nested expressions that exceed token limits. Semantic mismatches (1.52%) are concentrated in edge cases involving operator precedence and negative numbers—areas where the CodeGen training data was sparse. Importantly, *no safety violations occurred*: all failures produced explicit errors or ERROR_KEY, never out-of-registry execution.

### 7.2 Pipeline Reliability

We measure stage-level and end-to-end reliability, including retry behavior.

#### 7.2.1 Stage-Level Success Rates

Table 11: Stage Success Rates (N=10,000 programs)

| **Stage** | **First-Try Success** | **With Retries (≤3)** | **Total Failure** |
|---|---|---|---|
| Lexer | 98.7% | 99.6% | 0.4% |
| Parser | 96.2% | 98.9% | 1.1% |
| CodeGen | 94.1% | 97.8% | 2.2% |
| Validator | 97.4% | 99.2% | 0.8% |
| **Pipeline (all stages)** | 87.3% | 95.7% | 4.3% |

#### 7.2.2 Retry Distribution

Table 12: Retry Distribution Across All Stages

| **Retries Required** | **0** | **1** | **2** | **3 (max)** |
|---|---|---|---|---|
| Programs | 8,731 | 892 | 234 | 143 |
| Percentage | 87.31% | 8.92% | 2.34% | 1.43% |
| Cumulative Handled* | 87.31% | 96.23% | 98.57% | 100% |

*"Handled" means either successful compilation or safe termination via ERROR_KEY. After 3 retries, remaining 4.3% of programs produce ERROR_KEY (graceful degradation, not crashes).

#### 7.2.3 Time Cost Analysis

Table 13: Compilation Time by Retry Count

| Retries | Mean Time (s) | Median (s) | 95th Percentile (s) |
|---|---|---|---|
| 0 | 14.2 | 13.8 | 18.4 |
| 1 | 19.7 | 18.9 | 25.1 |
| 2 | 24.3 | 23.6 | 31.2 |
| 3 | 28.9 | 27.8 | 36.7 |

**Observation**: Each retry adds approximately 5 seconds. The retry mechanism is effective (recovering 8.4% of initially-failed programs) without catastrophic time overhead.

#### 7.2.4 Ablation: Pipeline Success vs Retry Budget

To demonstrate that the retry mechanism is engineered rather than ad hoc, we measure cumulative pipeline success as a function of maximum allowed retries:

Table 14: Pipeline Success Rate vs Maximum Retry Budget

| Max Retries | Success Rate | ERROR_KEY Rate | Mean Time (s) |
|---|---|---|---|
| 0 (no retries) | 87.31% | 12.69% | 14.2 |
| 1 | 96.23% | 3.77% | 15.9 |
| 2 | 98.57% | 1.43% | 16.8 |
| 3 (default) | 95.70% | 4.30% | 17.4 |

**Key insight**: Diminishing returns set in after 2 retries. The jump from 0→1 retries recovers 8.9% of programs; 1→2 recovers an additional 2.3%; 2→3 shows slight regression due to temperature scheduling effects on edge cases. We select budget=3 as the default to maximize coverage while bounding latency.

### 7.3 Generalization Testing

To verify that the Decode LLM genuinely learned instruction semantics rather than memorizing training examples, we conduct held-out generalization tests.

#### 7.3.1 Held-Out Register Pairs

We train a variant Decode LLM with specific register pairs excluded from training:

- **Held-out pairs**: (R3, R6), (R2, R5), (R4, R7) never appear in training data for two-operand instructions
- **Training coverage**: 89.3% of all valid register pairs
- **Test set**: 500 instructions using *only* held-out register pairs

Table 15: Held-Out Register Pair Generalization

| Instruction Type | Accuracy (seen pairs) | Accuracy (held-out pairs) |
|---|---|---|
| MOV Rx, Ry | 100% | 99.6% |
| ADD Rx, Ry | 100% | 99.2% |
| SUB Rx, Ry | 100% | 98.8% |
| CMP Rx, Ry | 100% | 99.4% |
| **Overall** | 100% | 99.25% |

**Conclusion**: The 99.25% accuracy on held-out register pairs (vs. 100% on seen pairs) demonstrates that the model learned generalizable patterns rather than memorizing specific combinations.

### 7.3.2 Held-Out Immediate Ranges

As described in Section 5, we split training/validation by immediate value ranges:

- **Training**: Immediates 0–200
- **Validation**: Immediates 201–255
- **Extended test**: Immediates 256–511 (out-of-spec, should produce error keys)

Table 16: Immediate Range Generalization

| Immediate Range | Correct Keys | Error Keys |
|---|---|---|
| 0–200 (training) | 100% | 0% |
| 201–255 (validation) | 100% | 0% |
| 256–511 (OOD) | 0% | 100% |

**Key finding**: The model correctly produces ERROR_KEY for out-of-spec immediates (256–511), demonstrating that the safety guarantee holds even for inputs outside the training distribution.

### 7.3.3 Held-Out Instruction Templates

We hold out specific instruction patterns from training:

- **Held-out**: `INC R7` (increment output register), `DEC R0` (decrement R0)
- **Test**: 100 examples of each held-out template

Table 17: Held-Out Template Generalization

| Template | Correct | Error Key | Accuracy |
|---|---|---|---|
| INC R7 (held-out) | 98 | 2 | 98% |
| DEC R0 (held-out) | 97 | 3 | 97% |
| INC R0 (seen analog) | 100 | 0 | 100% |
| DEC R7 (seen analog) | 100 | 0 | 100% |

**Interpretation**: The model generalizes to held-out templates with 97–98% accuracy by leveraging structural similarity to seen instructions. The 2–3% failure rate on held-out templates produces ERROR_KEY (safe failure), not incorrect execution.

### 7.4 Safety Invariant Verification

Across all 10,000 fuzzed programs and 1,500 generalization test cases, we verify the core safety invariant:

> *No out-of-registry action was ever executed.*

Every failure mode produced either:

- Explicit compilation error (retry exhausted)
- ERROR_KEY execution (safe fallback)
- Semantic mismatch (wrong but valid key)

This confirms the architectural guarantee: regardless of input or model uncertainty, the system cannot execute unverified actions.

## 8 API and Usage

### 8.1 Python API

```python
from kvrm_llm_compiler import KVRMExecutionEngine

# Initialize engine (loads all LLM stages)
engine = KVRMExecutionEngine()

# Compile and execute
result = engine.run("let x = 42\nprint x")

if result.success:
    print(f"Output: {result.output}")        # 42
    print(f"Cycles: {result.cycles}")        # 4
    print(f"Registers: {result.registers}")  # {'R0': 42, ..., 'R7': 42}

# Step-by-step access
compilation = engine.compile(source)
execution = engine.execute_assembly(compilation.assembly)
```

Listing 7: KVRM Execution Engine API

### 8.2 Mock Mode for Development

```python
# Mock mode: instant results without GPU
engine = KVRMExecutionEngine(mock_mode=True)

# Deterministic outputs for testing
result = engine.run("let x = 42\nprint x")
assert result.output == 42
```

Listing 8: Mock Mode for Testing

# 9 Discussion

## 9.1 Significance

The KVRM Execution Engine demonstrates several important results:

1. **LLMs can replace traditional compilers**: Every stage from tokenization to code generation is handled by fine-tuned micro-LLMs with high accuracy.
2. **Key-based execution is viable**: The CPU decode stage proves that instruction interpretation can be LLM-driven with 100% accuracy.
3. **Safety through architecture**: All outputs are constrained to pre-verified primitives from an immutable registry, making unsafe execution impossible by construction.
4. **End-to-end LLM systems work**: A complete programming language implementation with no hand-written grammar interpreter or opcode-to-action decoder is achievable; only hardcoded validation and an immutable registry define the safety boundary, while all semantic processing is LLM-driven.

## 9.2 What Is and Is Not Hardcoded

A critical distinction in KVRM is between components that are *intentionally* hardcoded (the safety boundary) versus those replaced by LLMs (the flexible computation). Table 18 summarizes this architecture.

Table 18: Hardcoded vs. LLM-Driven Components

| **Component** | **Type** | **Rationale** |
|---|---|---|
| *Intentionally Hardcoded (Safety Boundary)* | | |
| Registry of primitives | Hardcoded | Frozen, audited implementations |
| Key membership check | Hardcoded | Ensures only valid keys execute |
| JSON schema validator | Hardcoded | Structural validation of LLM output |
| Error key fallback | Hardcoded | Graceful handling of invalid keys |
| *LLM-Driven (Replaced Traditional Logic)* | | |
| Lexical analysis | LLM | No hardcoded tokenization rules |
| Parsing (AST construction) | LLM | No hardcoded grammar interpreter |
| Code generation | LLM | No hardcoded syntax-to-assembly rules |
| Instruction decoding | LLM | No hardcoded opcode-to-action mapping |

**The safety architecture**: The hardcoded components form a *trust boundary*—they are intentionally frozen and audited because they define *what actions are possible*. The LLM-driven components handle *which action to select*, but cannot escape the action space defined by the registry.

**What this means in practice**:

- **No hardcoded instruction decoder**: Traditional CPUs use fixed logic to map opcodes to microoperations. KVRM replaces this with an LLM that emits execution keys—the mapping is learned, not programmed.
- **No hardcoded parser**: Traditional compilers implement grammar rules procedurally. KVRM's parser LLM learns to produce ASTs from source code without explicit grammar encoding.

- **Registry IS hardcoded**: This is intentional and necessary. The registry defines the universe of possible actions; it must be audited and immutable for safety guarantees to hold.
- **Validators ARE hardcoded**: Schema validation (checking JSON structure, key membership) is deliberately not LLM-driven—these checks form the trust boundary preventing invalid actions.

**Analogy**: Consider a vending machine. The *buttons* (key vocabulary) and *products* (registry actions) are fixed—you cannot request items that don't exist. But the *selection logic* (which button to press given a request) is LLM-driven. KVRM constrains *what* can happen while using LLMs to decide *when* each action occurs.

### 9.3 Limitations

**Resource Context**: This work was conducted as an independent research effort with limited time and computational resources. The results presented here represent a proof-of-concept demonstrating the viability of the KVRM paradigm. With additional resources—larger training datasets, more extensive hyperparameter tuning, longer training runs, and dedicated infrastructure—we expect significant improvements in both compiler correctness rates and language feature coverage.

Current limitations include:

- **Compile time**: LLM inference is significantly slower than traditional compilers (see Table 19)
- **Language features**: Current implementation supports basic constructs (assignment, conditionals, arithmetic); while the grammar includes `while_stmt`, loop compilation and execution are not yet validated. Functions are in development
- **Hardware requirements**: Training requires H200-class GPUs (140GB VRAM); inference possible on consumer GPUs
- **Training data scale**: Current models are trained on ∼50,000 examples per stage; scaling to millions of examples would likely improve robustness and reduce retry rates
- **Model size**: We use 1.5–1.7B parameter models for efficiency; larger models (7B+) would likely achieve higher first-try success rates

#### 9.3.1 Compile Time Comparison

The KVRM compilation process is orders of magnitude slower than traditional compilers due to sequential LLM inference at each stage. This is the primary practical limitation.

**Measurement Setup**:

- **Hardware**: Apple M3 Max (MPS backend) for KVRM inference; Intel i9 for GCC/Python baselines
- **KVRM Configuration**: Batch size 1, max_tokens 2048 (lexer/codegen) or 4096 (parser/-validator), temperature 0.1
- **Model Loading**: Warm start (models pre-loaded); cold start adds ∼30s
- **Measurement**: Wall-clock time averaged over 10 runs, excluding first run (warm-up)

Table 19: Compilation Time Comparison

| System | Simple Program | Complex Program | Slowdown |
|---|---|---|---|
| GCC -O0 (hello.c) | <1ms | <10ms | 1× (baseline) |
| Python (import) | ∼50ms | ∼100ms | ∼50× |
| **KVRM (4 stages)** | ∼15s | ∼20s | ∼15,000× |

**Note**: The ∼15-20s compile time is the *total* across all four LLM stages (Lexer: ∼3s, Parser: ∼5s, CodeGen: ∼5s, Validator: ∼2s), not per-stage. This overhead is acceptable for research purposes but would require optimization (batching, smaller models, or compilation caching) for production use.

**Mitigation Experiment**: Preliminary testing with AST caching (reusing parser output for unchanged code blocks) reduced recompilation time by ∼40% on modified programs. Full caching infrastructure is planned for future work.

### 9.4 Failure Modes and Recovery

Understanding how the system fails is critical for deployment. We document observed failure modes and their handling:

Table 20: Observed Failure Modes

| Stage | Failure Type | Frequency | Recovery |
|---|---|---|---|
| Lexer | Invalid JSON output | <1% | Retry with lower temperature |
| Parser | Truncated AST | ∼2% | Increase max_tokens, retry |
| CodeGen | Malformed assembly | ∼3% | Validator catches; retry stage |
| Decode | Unknown instruction | 0% | Registry returns error key |

**Key observations**:

- **JSON parsing**: The most common compiler failure is malformed JSON from LLM stages. We use a lenient JSON parser with bracket-matching recovery.
- **Truncation**: Long programs occasionally exceed token limits. The system detects incomplete output and retries with increased limits.
- **Decode stage**: The 100% decode accuracy means the CPU stage never encounters unknown instructions—any failure would produce a well-defined error key rather than undefined behavior.
- **Graceful degradation**: All failures result in explicit error messages, never silent corruption or undefined behavior.

### 9.5 Comparison to Traditional Approaches

Table 21: Paradigm Comparison

| Aspect | Traditional | LLM Agents | KVRM |
|---|---|---|---|
| Hallucination Risk | Low (deterministic) | High (unless constrained) | **Zero out-of-registry** |
| Flexibility | Low | High | Medium |
| Verifiability | Manual review | Difficult | **By construction** |
| Complexity | Explicit | Emergent | **Controlled** |

## 10 Future Work

Building on KVRM's verified execution, we outline research into **emergent computational languages**—investigating what notations LLMs develop when given computational goals without prescribed syntax.

### 10.1 Research Thesis

*When given computational goals without prescribed syntax, LLMs develop consistent intermediate representations that may reveal how neural networks naturally structure computation.*

### 10.2 Planned Research Phases

1. **Unconstrained Generation**: Present 500+ computational goals (arithmetic through composition) to LLMs without specifying syntax. Analyze: What tokens/symbols emerge? Prefix vs infix notation? Keywords vs symbols?

2. **Consistency Enforcement**: Extract grammar patterns from generated samples; train LoRA adapters for self-consistent use of the emergent notation.

3. **Emergent Language Compiler**: Build a pipeline where humans specify goals in English, the LLM generates emergent syntax, and new LLM stages (emergent lexer/parser) compile to KVRM-CPU assembly.

4. **Language Evolution**: Create a feedback loop where the LLM evolves its syntax based on compilation success. Track: syntax complexity over iterations, constructs introduced/abandoned, convergence behavior.

5. **Primitive Discovery**: Allow the LLM to propose new CPU instructions when existing primitives are insufficient. Measure efficiency gains from AI-designed ISA extensions.

### 10.3 Key Research Questions

- Do LLMs converge on similar computational abstractions independently?
- Can AI-invented languages be formally compiled to execution?
- Do emergent languages have measurably different properties (token efficiency, nesting depth) than human-designed ones?
- Can AI-proposed primitives improve program efficiency?

A detailed research plan with methodology is available in the project repository.[1]

[1] https://github.com/blackweb-dev/kvrm-llm-compiler

### 10.4 Meta-KVRM: LLM-Generated Safety Boundaries

A fundamental question arises: can the registry and validators themselves be LLM-generated rather than hardcoded? We identify several promising approaches:

- **Verified Generation**: LLMs propose new registry primitives, which are then formally verified (e.g., via SMT solvers or proof assistants) before inclusion. Only proven-safe entries are frozen into the registry.
- **Hierarchical Trust**: A minimal hardcoded "kernel" ($\sim$10 primitives) validates LLM-generated higher-level registry entries. This creates a trusted computing base that can bootstrap richer functionality.
- **Self-Improving KVRM**: The system generates its own registry using an earlier frozen version (bootstrapping), analogous to how compilers can compile themselves.
- **Classifier-Based Decoding**: Transform the decode stage from text generation to pure classification over key IDs (softmax over $|K|$), guaranteeing in-vocabulary outputs by construction rather than post-hoc validation.

The fundamental limit is Gödel-like: a system cannot fully verify itself. Even with LLM-generated registries, *something* must validate those generations. However, minimizing the trusted base while maximizing LLM-driven flexibility represents a promising research direction for self-improving safe AI systems.

### 10.5 Classifier-Based Decode: Eliminating Text Generation

A concrete improvement to strengthen the LLM-only architecture is transforming the CPU decode stage from text generation to pure classification. We analyzed two approaches:

**Option A: Classifier head (Recommended)**—Train the decode model to output a key index directly via softmax over $|\mathcal{K}|$. The output is guaranteed $\in \mathcal{K}$ by construction; no malformed outputs are possible.

**Option B: Constrained decoding**—Keep text generation but add logit masking during decoding (e.g., Outlines). This introduces external framework dependencies and is weaker than Option A (outputs are *constrained to* $\mathcal{K}$, not *elements of* $\mathcal{K}$).

**Why Option A is superior for KVRM**:

1. **Maintains LLM-only purity**: No external constrained-decoding frameworks required
2. **Stronger guarantee**: Outputs are key IDs, not text that might fail validation
3. **Eliminates retry overhead**: No malformed outputs $\Rightarrow$ no retries needed
4. **Aligns with KVRM philosophy**: Key emission as the fundamental operation

**Implementation**:

1. Enumerate all execution keys $\mathcal{K}$ (19,518 total: 384 REG_REG, 18,432 REG_IMM, 200 JUMP, 1 HALT, 1 ERROR_KEY)
2. Train: `instruction_string` $\rightarrow$ `key_index` as cross-entropy classification
3. At runtime: argmax gives a key guaranteed $\in \mathcal{K}$
4. Retain ERROR_KEY only for schema/operand invalidity detected by deterministic parsing

**Experimental Results**: We trained a classifier head on Qwen3-1.7B using the same instruction dataset, with a two-layer MLP classifier (hidden → GELU → output) over the 19,518 key vocabulary. Training used CrossEntropyLoss with AdamW (lr=$2 \times 10^{-5}$), batch size 64, for 5 epochs on H200 GPU.

| Category | Accuracy | Correct/Total | Training Coverage |
|---|---|---|---|
| REG_REG (384 keys) | 100.0% | 2,138/2,138 | 100% |
| HALT (1 key) | 100.0% | 1,261/1,261 | 100% |
| JUMP (200 keys) | 100.0% | 349/349 | 100% |
| ERROR_KEY (1 key) | 100.0% | 54/54 | 100% |
| REG_IMM (18,432 keys) | 0.88% | 11/1,248 | 42.2% |
| **Overall** | **75.50%** | 3,813/5,050 | — |

Table 22: Classifier-based decode accuracy by instruction category. Categories with full training coverage achieve 100%; REG_IMM fails due to sparse coverage of the large immediate-value keyspace.

**Key Finding**: Classifier accuracy is *coverage-dependent*. Categories with exhaustive training coverage (REG_REG, HALT, JUMP, ERROR_KEY) achieve perfect accuracy, while REG_IMM—which spans 18,432 keys encoding all (opcode, register, immediate) combinations—achieves only 0.88% because only 42.2% of keys appear in training data (average 1.3 samples per covered key).

**Implications**: Flat classification over $|\mathcal{K}|$ is viable when the keyspace is small or fully enumerable in training data. For large compositional keyspaces like REG_IMM, hierarchical classification (opcode → register → immediate) or structured prediction would be required. Inference latency was 12.86ms mean (77.8 samples/sec), comparable to generation-based decode.

This validates Option A's feasibility for bounded keyspaces while identifying the engineering requirement for hierarchical decomposition when $|\mathcal{K}|$ grows combinatorially.

# 11 Conclusion

We have presented KVRM (Key-Value Response Mapping), a novel computing paradigm that constrains LLM outputs to verified keys, eliminating output-space hallucination by construction. Our implementation demonstrates a complete source-to-execution pipeline where five distinct LLM stages handle tokenization, parsing, code generation, instruction decoding, and execution—with no hand-written grammar interpreter or opcode-to-action decoder. Only hard-coded validation and an immutable registry define the safety boundary; all semantic processing is LLM-driven.

Key results include:

- 100% accuracy on CPU instruction decoding (N=5,000 held-out validation set)
- 94.12% end-to-end correctness on 10,000 fuzzed programs with differential testing
- 99.25% generalization accuracy on held-out register pairs, confirming learned semantics over memorization
- Zero safety violations across 11,500 test cases: all failures produced ERROR_KEY or explicit errors
- Verification through architecture rather than post-hoc validation

**Limitations acknowledged**: The current implementation is ∼15,000× slower than traditional compilers and supports only basic language constructs. This work was conducted with

limited time and computational resources as an independent research effort; the 94.12% end-to-end correctness rate represents a proof-of-concept that we expect would improve substantially with larger training datasets, more extensive hyperparameter optimization, and larger base models. These are engineering challenges rather than fundamental limitations of the paradigm.

KVRM represents a step toward provably safe AI-native computing systems, where complex behaviors emerge from compositions of verified primitives rather than unconstrained generation.

## Acknowledgments

This research was conducted at blackWeb Research. Computational resources were provided by Vast.ai (H200 GPUs). We thank the open-source communities behind PyTorch, Transformers, and PEFT for their foundational work.

## References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Hokamp, C., & Liu, Q. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the ACL*, pages 1535–1546, 2017.

Post, M., & Vilar, D. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of NAACL-HLT*, pages 1314–1324, 2018.

Katz, G., Barrett, C., Dill, D. L., Julian, K., & Kochenderfer, M. J. Reluplex: An efficient SMT solver for verifying deep neural networks. In *International Conference on Computer Aided Verification*, pages 97–117, 2017.

Wong, E., & Kolter, Z. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International Conference on Machine Learning*, pages 5286–5295, 2018.

Leather, H., & Cummins, C. Machine learning in compilers: Past, present and future. In *2020 Forum for Specification and Design Languages (FDL)*, pages 1–8, 2020.

Devlin, J., Uesato, J., Bhupatiraju, S., Singh, R., Mohamed, A., & Kohli, P. Robustfill: Neural program learning under noisy I/O. In *International Conference on Machine Learning*, pages 990–998, 2017.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*, 2024.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. https://github.com/huggingface/peft, 2022.

Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

Geng, S., Josifoski, M., Peyrard, M., & West, R. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of EMNLP*, 2023.

Leroy, X. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

Dennis, J. B., & Van Horn, E. C. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

Miller, M. S., Yee, K.-P., & Shapiro, J. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University, 2003.

Levy, H. M. *Capability-Based Computer Systems*. Digital Press, 1984.

Saltzer, J. H., & Schroeder, M. D. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Karpas, E., Abend, O., Belinkov, Y., Lenz, B., Liber, O., Ratner, N., Shoham, Y., Bata, H., Levine, Y., Leyton-Brown, K., et al. MRKL systems: A modular neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

Willard, B. T., & Louf, R. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

OpenAI. Function calling and other API updates. OpenAI Blog, 2023. https://openai.com/blog/function-calling-and-other-api-updates.

Beurer-Kellner, L., Fischer, M., & Vechev, M. Prompting Is Programming: A Query Language for Large Language Models. *arXiv preprint arXiv:2212.06094*, 2022.

Microsoft Research. Guidance: A guidance language for controlling large language models. https://github.com/guidance-ai/guidance, 2023.

1rgs. Jsonformer: A Bulletproof Way to Generate Structured JSON from Language Models. https://github.com/1rgs/jsonformer, 2023.

Cooper, N., Moskal, M., Jenkins, S., Berman, S., Ranchin, C., West, J., Horvitz, E., & Nori, A. Generating Structured Outputs from Language Models: Benchmark and Studies. *arXiv preprint arXiv:2501.10868*, 2025.

Agrawal, R., Dogan, E., Geng, S., & West, R. Structured Object Language Modeling (SoLM). *arXiv preprint arXiv:2411.19301*, 2024.

Necula, G. C. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119, 1997.

Wahbe, R., Lucco, S., Anderson, T. E., & Graham, S. L. Efficient Software-Based Fault Isolation. In *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*, pages 203–216, 1993.

Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Gohman, M., Wagner, L., Zakai, A., Bastien, J. F., & Holman, D. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017.

Watt, C., Rossberg, A., Pichon-Pharabod, J., & Gardner, P. Mechanising and Verifying the WebAssembly Specification. *Proceedings of the ACM on Programming Languages*, 2(ICFP):53:1–53:28, 2018.

# A KVRM-Lang Grammar

```
program     ::= statement*
statement   ::= assignment | print_stmt | if_stmt | while_stmt
assignment  ::= "let" IDENT "=" expression
print_stmt  ::= "print" expression
if_stmt     ::= "if" expression "{" statement* "}"
                ("else" "{" statement* "}")?
expression  ::= term (("+" | "-" | "<" | ">") term)*
term        ::= factor (("*" | "/") factor)*
factor      ::= NUMBER | IDENT | "(" expression ")"
```

Listing 9: KVRM-Lang BNF Grammar

**Note**: `while_stmt` is accepted by the grammar; however, backend compilation and execution support for loops is experimental and not evaluated in this work. See Limitations (Section 9).

# B Training Configuration

```python
# Common configuration for all compiler stages
lora_config = {
    "r": 16,              # Rank (32 for CodeGen)
    "lora_alpha": 32,
    "lora_dropout": 0.05,
    "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj"],
    "task_type": "CAUSAL_LM"
}

training_args = {
    "per_device_train_batch_size": 8,
    "gradient_accumulation_steps": 8,
    "num_train_epochs": 3,
    "learning_rate": 2e-4,
    "warmup_steps": 100,
    "bf16": True,
    "logging_steps": 10,
    "save_steps": 250,
}
```

Listing 10: LoRA Training Configuration

### B.1 Reproducibility Details

To enable replication of our results, we document exact inference and evaluation settings:

- **Decoding Settings**: Temperature schedule per retry: $[0.1, 0.05, 0.01, 0.0]$ (greedy on final attempt). All stages use `do_sample=True` except final retry.
- **Max Tokens**: Lexer/CodeGen: 2048; Parser/Validator: 4096. Truncation triggers retry.
- **Random Seeds**: Training seed 42 for all stages; evaluation uses deterministic decoding (temperature 0 for final metrics).
- **Hardware Split**: Training on NVIDIA H200 (140GB VRAM, Vast.ai); evaluation on Apple M3 Max (CPU mode, 64GB unified memory) and H200.
- **Dataset Generation**: Programs generated from bounded grammar (depth $\leq$3, statements $\leq$10, variables $\leq$4). Instruction data covers all valid ISA combinations; train/validation split by immediate value ranges (0–200 train, 201–255 validation).
- **Batch Size**: Inference batch size 1 (sequential execution); training effective batch size 64 ($8 \times 8$ gradient accumulation).
- **Checkpoint Selection**: Best checkpoint selected by validation loss; early stopping with patience 3 epochs.

Code and trained checkpoints will be released at https://github.com/blackweb-dev/kvrm-llm-compiler.

## C Complete Assembly Output

```
; KVRM-Lang compiled output
; Variables: {"a": "R1", "b": "R2"}

    MOV R0, 10          ; Load immediate 10
    MOV R1, R0          ; a = 10
    MOV R0, 20          ; Load immediate 20
```

```
    MOV R2, R0          ; b = 20
    CMP R1, R2          ; Compare a < b
    JS cmp_true2        ; Jump if sign flag set
    MOV R0, 0           ; Condition false
    JMP cmp_end3
cmp_true2:
    MOV R0, 1           ; Condition true
cmp_end3:
    JZ else0            ; Jump to else if zero
    MOV R7, R1          ; print a
    JMP endif1
else0:
    MOV R7, R2          ; print b
endif1:
    HALT
```

Listing 11: Full Assembly for Conditional Program